

Examen réparti 2
Master 1 SAR
UE 4I401

L. Arantes, P. Cadinot, S. Dubois, J. Lejeune, J. Sopena, P. Sens

Janvier 2018

2 heures – Tout document papier autorisé
Tout appareil électronique interdit
Le barème sur 21,5 points est donné à titre indicatif

Exercice 1 – Questions de cours (2,5 points)

Question 1 $\frac{1}{2}$ point

Qu'est-ce qu'un noyau non-préemptif ? Le noyau du TD est-il préemptif ?

Solution:

Dans noyau non-préemptif on perd la main dans le noyau qu'en cas de blocage explicite. Le noyau est non préemptif.

Question 2 $\frac{1}{2}$ point

Lors d'un appel système, sur quelle pile s'exécute le code noyau de cet appel ?

Solution:

sur la pile système du processus courant

Question 3 $\frac{1}{2}$ point

Quel processus crée le processus init ? Indiquez le pid de ce processus.

Solution:

Le processus 0

Question 4 $\frac{1}{2}$ point

A quel moment l'inode d'un fichier de données est ramenée en mémoire ?

Solution:

Lors d'un open J'ai mis ok aussi si on fait un lget

Question 5 $\frac{1}{2}$ point

L'inode d'un même fichier peut-elle être à un même moment plusieurs fois en mémoire ? Justifiez votre réponse.

Solution:

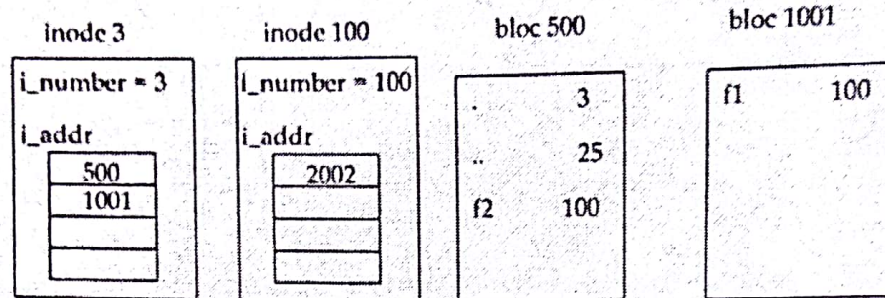
Non, si il y a plusieurs open, seul le `i_count` est incrémenté

Exercice 2 – Système de fichiers et Buffer Cache (6 points)

Les *inodes* ont une structure équivalente à celle vue en TD.

Soit le fichier `f1` dont le contenu est le suivant : "Examen"

Le répertoire courant correspond à l'inode numéro 3. Nous considérons les *inodes* et les blocs suivants :



Le bloc 500 ne contient pas la chaîne de caractère "f1".

Soit le code utilisateur suivant :

```
1 int f1, f2;
2 char buffer[16];
3 f1 = open("./f1", O_WRONLY | O_APPEND); /* O_APPEND positionne en
    fin de fichier */
4 f2 = open("./f2", O_RDONLY);
5 if (fork() == 0) {
6     write(f1, "Final", 5);
7     close(f1); close(f2);
8     exit(0);
9 }
10 wait(NULL);
11 write(f1, "2018", 4);
12 read(f2, buffer, 15);
13 buffer[15] = 0;
14 printf("%s", buffer);
15 close(f1);
16 close(f2);
17 exit(0);
```

Question 1 1 point

Quel est l'affichage produit par le `printf` de la ligne 14 compte tenu des indications données sur les inodes et les blocs (justifiez)?

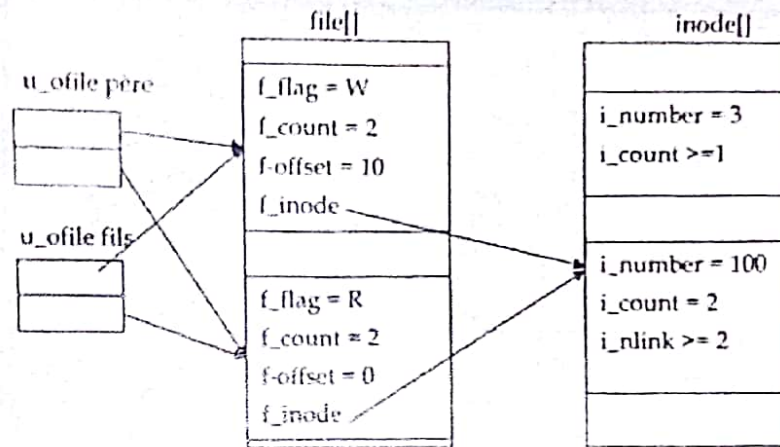
Solution:

ExamenFinal2018

Question 2 1 1/2 points

Donnez l'état, juste après la ligne 6, des tables de descripteurs de fichiers, fichiers ouverts, et inodes (faites un schéma, pensez à représenter notamment les champs `f_count`, `f_offset`, `f_flags`, `i_count`, `i_nlink` et `i_number`).

Solution:



Petite erreur sur le schéma : le `f_offset` du premier file est égal à 11 et non 10

Question 3 2 points

Avant l'exécution de ce code, nous faisons l'hypothèse que l'inode du répertoire courant (inode numéro 3) et l'inode 100 sont présentes dans la table `inode[]`. Donnez la séquence des blocs et des inodes accédés jusqu'à la fin de la ligne 6. Pour chaque accès, spécifiez la ligne de code associée.

Solution:

3- open - namei

inode 3 en mémoire

accès bloc 500

accès bloc 1001

accès inode 100 en mémoire

4- open - namei

inode 3 en mémoire

accès bloc 500

accès inode 100 en mémoire

6- write

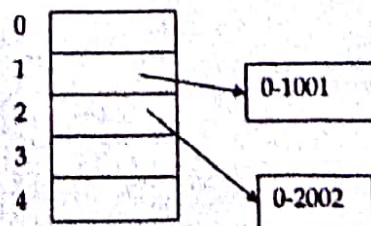
accès bloc 2002

séquence d'accès aux blocs ; 500, 1001, 500, 2002

séquence d'accès aux inodes : 3,100,3,100

Question 4 1½ points

Soit la configuration suivante du buffer cache avant l'exécution du code. Le buffer cache ne peut contenir au maximum que 3 blocs. Les numéros de bloc sont représentés sous la forme `devno-blockno`. La fonction de hachage associant un bloc à une des sous-listes est `devno+blockno % 5`. Nous considérons que pour tous les blocs `devno = 0`.



La bfreelist est la suivante (de la tête vers la queue) : 0-1001, 0-2002. Initialement aucun des blocs n'est marqué avec le flag DELVRI et les writes font des écritures différée. Donnez la configuration du buffer cache après la ligne 6 du code. Quel est le nombre d'entrées/sorties physiques ?

Solution:

Séquence : 0-500, 0-1001, 0-500, 0-2002

Accès : bfreelist

0-500 : 0-1001, 0-2002, 0-500 (1 E/S)

0-1001 : 0-2002, 0-500, 0-1001

0-500 : 0-2002, 0-1001, 0-500

0-2002 : 0-1001, 0-500, 0-2002

1 E/S physique

Exercice 3 – Swap (4 points)

Question 1 1 point

Rappelez le rôle des variables runin et runout.

Solution:

Ce sont deux variables globales sur lequel le swapper s'endort (`runout++`; `sleep(&runout)` et `runin++`; `sleep(&runin)`)

Il s'endort sur la variable :

- `runout` lorsqu'il n'a pas trouvé de processus à ramener en mémoire (un processus SRUN et non chargé)
- `runin` lorsqu'il n'a pas trouvé de processus en mémoire éligible à être swapper

Question 2 1/2 point

Donnez une raison pour faire un wakeup sur runout

Solution:

Un wakeup sur runout se fait :

- lorsqu'un processus passe à l'état RUN mais n'est pas chargé en mémoire (fonction `setrun`)
- lors d'un autoswap (fonction `xswap`)

Question 3 1/2 point

Donnez une raison pour faire un wakeup sur runin

Solution:

Un wakeup sur runin se fait :

- toutes les secondes (fonction `realtime`)
- lorsqu'un processus s'endort (fonction `sleep`)

On considère que la mémoire comporte 15 unités et la table proc ne comporte que 5 processus. Voici l'état de cette table juste après l'exécution de la fonction realtime où p_size est exprimé en nombre d'unités :

p_pid	p_flag	p_stat	p_size	p_time	p_wchan
0	SSYS SLOAD	SSLEEP	2	120	&runout
1	SSYS SLOAD	SRUN	4	115	
255	SLOAD	SRUN	5	1	
256	SLOAD	SSLEEP	4	6	&toto
257	0	SSLEEP	6	5	

en memoire

s'endort
carout

On fait les hypothèses suivantes :

- Aucun fork ni exit, ni wakeup(&toto) ne sera appelé par la suite.
- Le temps d'attente de fin d'entrée/sortie est considéré négligeable par rapport à une seconde.
- On ne prendra pas en compte les segments texte des processus.

Question 4 2 points

Le processus 257 passe à l'état SRUN, donnez l'évolution de la table jusqu'à ce qu'il soit chargé en mémoire. Si besoin vous pouvez considérer un ou plusieurs appels à realtime. Pour simplifier, vous pouvez lister uniquement la liste des champs qui changent.

Solution:

Jusqu'au prochain appel à realtime :

Donc après le passage à l'état SRUN entraîne :

1. switch vers le swapper immédiatement car il est endormi sur runout (wakeup dans le sleep) et qu'il a la plus forte priorité.
2. le swapper choisi 257 pour le sortir du swap car ce dernier est SRUN mais pas SLOAD
3. impossible de charger 257 car la mémoire est pleine, il choisit donc de swapper 256 car à l'état sleep ($\Rightarrow$ I/O pour swapper 256)
4. une fois l'I/O terminée, le swapper se réveille dans xswap : on enlève le SLOAD de 256, son p_time passe à 0
5. il rechoisit 257 $\rightarrow$ toujours pas de place en mémoire et aucun processus à évincer (tous SSYS ou bien avec un p_time < 2) : le swapper s'endort donc sur runin

On a donc

p_pid	p_flag	p_stat	p_size	p_time	p_wchan
0	SSYS SLOAD	SSLEEP	2	120	&runin
1	SSYS SLOAD	SRUN	4	115	
255	SLOAD	SRUN	5	1	
256	0	SSLEEP	4	0	&toto
257		SRUN	6	5	

incrémenté
realtime

Jusqu'à l'appel à realtime suivant :

1. realtime s'exécute : tous les p_time sont incrémentés et le swapper est réveillé car on fait un wakeup sur runin
2. le swapper choisit 257 pour le sortir du swap (le seul à être SRUN)
3. pas de place en mémoire mais il est possible d'évincer 255 car son p_time est maintenant ≥ 2 ($\Rightarrow$ I/O pour swapper 255)
4. une fois l'I/O terminée, le swapper se réveille dans xswap : on enlève le SLOAD de 255, son p_time passe à 0

5. il choisit 257 et a désormais la place pour le charger en mémoire ($\Rightarrow$ I/O pour charger 257)
6. une fois l'I/O terminée on ajoute SLOAD à 257, son p_time passe à 0
7. il recommence son algorithme et choisit 256 pour le ramener en mémoire mais il n'y a pas assez de place (il reste 3 unités disponibles) et aucun processus n'est éligible à revenir sur le swap : le swapper s'endort donc sur runin

On a donc :

p_pid	p_flag	p_stat	p_size	p_time	p_wchan
0	SSYS SLOAD	SSLEEP	2	121	&runin
1	SSYS SLOAD	SRUN	4	116	
255	$\emptyset$	SRUN	5	2 puis 0	
256		SSLEEP	4	1	&toto
257	SLOAD	SRUN	6	0	

$\rightarrow$ revient à 0
(swap)

Exercice 4 – Synchronisation (9 points)

On souhaite gérer un nouveau périphérique «Part» qui permet d'envoyer et de recevoir un octet sur une ligne série.

Dans le noyau, la variable globale `part` est utilisée pour la réception et l'envoi de caractère ainsi que pour le contrôle de la synchronisation.

```
struct {
    int flag; /* utilisé pour la synchronisation */
    char input; /* octet reçu */
    char output; /* octet à envoyer */
} part;
```

Les flags suivants pourront être utilisés :

```
#define P_BUSY 0x0001 /* Envoi en cours */
#define P_DONE 0x0002 /* Reception terminée */
```

La fonction interne `strategy_part(char *arg)` demande au périphérique d'envoyer le contenu désigné par `arg` sur la ligne série. La fonction est asynchrone : elle retourne directement avant que l'envoi soit terminé sur le périphérique. Lorsque le périphérique a fini son envoi, il lève l'interruption numéro `IPART_SND` qui appelle la routine `anddone()` qui s'exécute au niveau `IPART_SND` du processeur.

Lorsque le périphérique reçoit un octet sur la ligne, il copie l'octet directement dans `part.input` et lève l'interruption `IPART_RCV` qui appelle la routine `rcvdone()` qui s'exécute au niveau `IPART_RCV` du processeur.

Question 1 2 points

On s'intéresse à la réception d'un octet sur la ligne. L'appel système `char recv()` attend la réception d'un nouvel octet sur la ligne. La fonction retourne l'octet reçu. Un octet doit être lu qu'une seule fois. Par exemple, si 2 processus appellent `recv()`, ils vont lire deux octets différents. En revanche, il est possible d'en perdre.

Donnez le code dans le noyau de l'appel système `_recv()` correspondant à `char recv()`. En cas de blocage la fonction s'endort à la priorité `PPART`.

Solution:

```
_rcv ( ) {  
    spl(IPART_RCV);  
    while ( (part.flag & P_DONE) == 0)  
        sleep(&part.input, PPART);  
    u.u_arg[0] = part.input ;  
    part.flag &= ~P_DONE;  
    spl(NORMAL);  
    return;  
}
```

Question 2 1½ points

Donnez le code de la routine rcvdone().

Solution:

```
rcvdone( ) {  
    part.flag |= P_DONE;  
    wakeup(&part.input);  
    return;  
}
```

Question 3 2 points

On s'intéresse maintenant à l'envoi d'un octet sur la ligne. L'appel système `void send(char c)` demande l'envoi de l'octet `c` sur la ligne. Attention : si un envoi est déjà en cours, la fonction doit se bloquer en s'endormant à la priorité `PPART`. Donnez le code dans le noyau de l'appel système `_send()` correspondant à `void send(char c)`.

Solution:

```
_send ( ) {  
    spl(IPART_SEND);  
    while ( (part.flag & P_BUSY) != 0)  
        sleep(&part.output, PPART);  
    part.flag |= P_BUSY;  
    part.output = u.u_arg[0];  
    spl(NORMAL);  
    strategy_part(&part.output);  
    return;  
}
```

Question 4 1½ points

Donnez le code de la routine snddone().

Solution:

```
snddone( ) {  
    part.flag &= ~P_BUSY;  
    wakeup(&part.output);  
    return;  
}
```

Question 5 2 points

Modifiez `_send()` pour que la fonction ne retourne que lorsque le périphérique a fini l'envoi de l'octet.

Solution:

```
_send ( ) {  
    spl(IPART_SND);  
    while ( (part.flag &P_BUSY) != 0)  
        sleep(&part.output,PPART);  
    part.flag |= P_BUSY;  
    part.output = u.u_arg[0];  
    spl(NORMAL);  
    strategy_part(&part.output);  
  
    /* Modifications : */  
    spl(IPART_SND);  
    while ( (part.flag &P_BUSY) != 0)  
        sleep(&part.output,PPART+1);  
    spl(NORMAL);  
    return;  
}
```